

Synchronized.pdf



JesusLagares



Programación Concurrente y de Tiempo Real



3º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería
Universidad de Cádiz**



[Accede al documento original](#)

antes



**Descarga sin publi
con 1 coin**



Después

WUOLAH



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Synchronized



`synchronized` es una palabra reservada que tenemos en Java para manipular el cerrojo inherente de un objeto.

Explicación

El concepto clave aquí es que **todo objeto en Java lleva incorporado un cerrojo (monitor) y una cola de espera asociada**, pero ese cerrojo **solo "existe" a efectos de concurrencia cuando se usa `synchronized`**.

El concepto es sencillo, pero debemos tener claras algunas características para que no nos la líen en el examen.

Un cerrojo tiene asociada una cola de exclusión mutua. Aunque estas colas no se crean hasta que se usa la palabra reservada.

Por lo que si hacemos algo como `int a` tenemos 0 colas; pero si se usa `synchronized(a)`, ya tenemos 1 cola.

El cerrojo de un objeto "se activa" cuando usamos `synchronized`

Un objeto normal, como:

```
int a;  
Object x = new Object();
```

no crea colas activas hasta que alguien ejecuta:

```
synchronized (x) { ... }
```

En ese momento empiezan a existir:

- **cola de exclusión mutua** (hilos que quieren entrar al monitor)

- **wait-set** (cola de hilos bloqueados con `wait()`)

Solo se pueden usar `wait()` , `notify()` y `notifyAll()` dentro de un bloque/método `synchronized`

En la API standart tenemos algunos métodos asociados al objeto para manipular la cola del wait-set. Estos métodos SOLO DEBEN USARSE DENTRO DE UN BLOQUE `synchronized` .

No es una restricción decorativa:

El compilador **no sabría qué cerrojo usar** si no estás dentro de un `synchronized` .

Además, solo puede llamar a `notify()` el hilo que **posee el cerrojo**, porque:

- Para hacer `wait()` , el hilo **libera el cerrojo**.
- Para hacer `notify()` , el hilo **debe tener el cerrojo**, es decir, estar en sección crítica.

Por lo tanto, el hilo que utilice esos métodos debe estar en exclusión mutua.

Para entender estos métodos es vital recordar cuántas colas tenemos en Java cuando hablamos de exclusión mutua y sincronización, y para qué es cada una.

Cola	Nombre	Qué representa
Cola del cerrojo	<i>entry set</i>	Hilos esperando a adquirir el cerrojo (exclusión mutua).
Wait-set	cola de condición	Hilos que han hecho <code>wait()</code> y están bloqueados.

Esto es materia de examen:

➡ *No tienen nada que ver entre sí.*

Un hilo puede estar esperando EM o esperando una condición.

Un objeto → un único cerrojo

Da igual cuántos métodos sincronizados haya sobre él.

```
synchronized(obj) {...}
synchronized void m() { ... } // usa this
```

Si ambos usan el **mismo objeto**, están usando **el mismo cerrojo**.

Si el método es **synchronized**, el cerrojo siempre es **this**

```
public synchronized void f() { ... }
```

Es equivalente a:

```
public void f() {  
    synchronized(this) { ... }  
}
```

Reentrancia del cerrojo

Los cerrojos *intrínsecos* de Java **son reentrantes**:

Si un hilo ya posee el cerrojo de un objeto y vuelve a intentar adquirirlo (por ejemplo, llamada recursiva o llamada a otro método sincronizado sobre el mismo objeto), **no se bloquea a sí mismo**.

Ejemplo:

```
public synchronized void a() {  
    b(); // también synchronized(this)  
}  
  
public synchronized void b() {  
    // no hay deadlock: el hilo ya posee the cerrojo  
}
```

Esto evita el clásico *self-deadlock*.